A Practical Guide to Rectifier neural networks

TOPIC -- RECTIFIER NEURAL NETWORKS

$$a^{(l)} = \sigma!\left(W^{(l)} a^{(l-1)} + b^{(l)}\right)$$

-- 01 --

f (x) = x Φ (x) = x · 1 2 [1 + erf ■ (x / 2)] , {\displaystyle f(x)=x\Phi (x)=x\cdot {\frac {1}{2}}\left[1+\operatorname {erf} \left(x/{\sqrt {2}}\right)\right],}

-- 02 --

This equation is foundational to the model's capacity; while the attention mechanism determines the relationships between tokens, the ReLU-based FFN performs the majority of the numerical computation and houses the bulk of the model's parameters. The efficiency and scalability of this rectified framework triggered a global technological revolution, enabling the development of Large Language Models that have had a profound economic impact. The industrial response to this architecture—including the massive expansion of AI-specific hardware and the birth of the generative AI sector—has positioned the Transformer as a cornerstone of 21st-century infrastructure. During the post 2017 period of rapid AI advancement, the rectified linear unit function has been key to achieving increased model performance and scaling due to the fact that it zeros out responses that are immaterial for a given stimuli, preventing them from accumulating in massive scale models. It is the complete silencing of the parts of the model found to be stimuli-irrelevant during learning that allows for scaling. As the stimuli-irrelevant proportion of the model becomes more massive, these highly numerous connections within the model would inevitably accumulate during scaling no matter how small each individual response is. Therefore, the rectified linear unit function, with its absolute zeroing property, enabled the scaling to hundred billion parameter models and beyond. Early Transformer scaling giants like GPT-3 (2020) and Falcon-180B (2023) relied on the rectified linear unit function explicitly, while successors such as GPT-4 (2023) and Llama 3 (2024) utilized smoother variants like GELU or SwiGLU. These variants were used to improve training stability while fundamentally preserving the rectified principle of zeroing low responses. At the centre of modern artificial intelligence ReLU and its variants maintain absolute zero response across the bulk of the

model at any one time, while maintaining approximately linear reponses for stimuli-relevant connections enabling high performance on each specific cognitive task. This feature of activation sparsity has been critical for massive scaling and performance gains of AI models right up to the present day.

-- 03 --

f (x) = { x x > x c , (e a x – 1) / b x ≤ x c f ′ (x) = { 1 x > x c , (a / b) e a x x ≤ x c {\displaystyle f(x)={\begin{cases}x&x>x_{c},\\(e^{ax}-1)/b&x\leq x_{c}\end{cases}}\qquad f'(x)={\begin{cases}1&x>x_{c},\\(a/b)e^{ax}&x\leq x_{c}\end{cases}}}

-- 04 --

f (x) = ln ■ (1 + e k x) k , f ′ (x) = e k x 1 + e k x = 1 1 + e – k x {\displaystyle f(x)={\frac {\ln(1+e^{kx})}{k}},\qquad f'(x)={\frac {e^{kx}}{1+e^{kx}}}={\frac {1}{1+e^{-kx}}}}

-- 05 --

ReLU ■ (x) = x + = max (0 , x) = x + | x | 2 = { x if x > 0 , 0 x ≤ 0 {\displaystyle \operatorname {ReLU} (x)=x^{+}=\max(0,x)={\frac {x+|x|}{2}}={\begin{cases}x&{\text{if }}x>0,\\0&x\leq 0\end{cases}}}

-- 06 --

In the context of artificial neural networks, the rectifier or ReLU (rectified linear unit) activation function is an activation function defined as the non-negative part of its argument, i.e., the ramp function:

-- 07 --

LSE ■ (x 1 , … , x n) = ln ■ (e x 1 + ■ + e x n) {\displaystyle \operatorname {LSE} (x_{1},\dots ,x_{n})=\ln(e^{x_{1}}+\cdots +e^{x_{n}})}

-- 08 --

f (x) ≈ 1 2 x (1 + tanh ■ [2 / π (x + 0.044715 x 3)]) {\displaystyle f(x)\approx {\frac {1}{2}}x\left(1+\tanh \left[{\sqrt {2/\pi }}\left(x+0.044715x^{3}\right)\right]\right)}

-- 09 --

and its gradient is the softmax; the softmax with the first argument set to zero is the multivariable generalization of the logistic function. Both LogSumExp and softmax are used in machine learning.

History The ReLU was first used by Alston Householder in 1941 as a mathematical abstraction of biological neural networks. Kunihiko Fukushima in 1969 used ReLU in the context of visual feature extraction in hierarchical neural networks. In 1998, Gregory Woodbury demonstrated that the rectified linear function could account for a broad range of emergent properties in the visual cortex. His work showed that a single unified model could drive the joint development of refined retinotopic maps, ocular dominance columns, and orientation selectivity. By utilizing the rectifier's "cutoff" property, Woodbury achieved a close quantitative fit to biological data, matching the spatial periodicities and topographic refinement patterns observed in macaque and cat cortical maps. Furthermore, he extended this framework to adult plasticity, accurately replicating the spatial and temporal dynamics of lesion-induced cortical reorganization. This research established that the rectified linear response was a necessary mechanism for the stable self-organisation and maintenance of complex, multi-feature neural maps. In 2000, Hahnloser et al. argued that ReLU approximates the biological relationship between neural firing rates and input current, in addition to enabling recurrent neural network dynamics to stabilise under weaker criteria. Prior to 2010, most activation functions used were the logistic sigmoid (which is inspired by probability theory; see logistic regression) and its more numerically efficient counterpart, the hyperbolic tangent. Around 2010, the use of ReLU became common again. Jarrett et al. (2009) noted that rectification by either absolute or ReLU (which they called "positive part") was critical for object recognition in convolutional neural networks (CNNs), specifically because it allows average pooling without neighboring filter outputs cancelling each other out. They hypothesized that the use of sigmoid or tanh was responsible for poor performance in previous CNNs. Nair and Hinton (2010) made a theoretical argument that the softplus activation function should be used, in that the softplus function numerically approximates the sum of an exponential number of linear models that share parameters. They then proposed ReLU as a good approximation to it. Specifically, they began by considering a single binary neuron in a Boltzmann machine that takes x {\displaystyle x} as input, and produces 1 as output with probability σ (x) = 1 1 + e − x {\displaystyle \sigma (x)={\frac {1}{1+e^{-x}}}} . They then considered extending its range of output by making infinitely many copies of it X 1 , X 2 , X 3 , … {\displaystyle X_{1},X_{2},X_{3},\dots } , that all take the same input, offset by an amount 0.5 , 1.5 , 2.5 , … {\displaystyle 0.5,1.5,2.5,\dots } , then their outputs are added together as ∑ i = 1 ∞ X i {\displaystyle \sum _{i=1}^{\infty }X_{i}} . They then demonstrated that ∑ i = 1 ∞ X i {\displaystyle \sum _{i=1}^{\infty }X_{i}} is approximately equal to N (log ■ (1 + e x) , σ (x)) {\displaystyle {\mathcal {N}}(\log(1+e^{x}),\sigma (x))} , which is also

approximately equal to ReLU ■ (N (x , σ (x))) {\displaystyle \operatorname {ReLU} ({\mathcal {N}}(x,\sigma (x)))} , where N {\displaystyle {\mathcal {N}}} stands for the gaussian distribution. They also argued for another reason for using ReLU: that it allows "intensity equivariance" in image recognition. That is, multiplying input image by a constant k {\displaystyle k} multiplies the output also. In contrast, this is false for other activation functions like sigmoid or tanh. They found that ReLU activation allowed good empirical performance in restricted Boltzmann machines. Glorot et al (2011) argued that ReLU has the following advantages over sigmoid or tanh: